

Stacking Handcrafted and Convolutional Features for Wafer Map Classification, Revisited

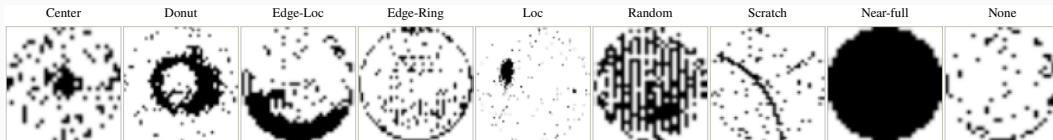
Reproduction, Meta-Training Pitfalls, and Improvements

Kartikeya Gupta

Indian Institute of Information Technology Sri City

The problem

- A **wafer map** records which dies on a wafer failed electrical tests.
- Failures form **patterns**; each points to a different fault in fabrication $\Rightarrow$ automatic classification supports yield analysis.
- WM-811K: 172,946 labeled wafers, 85% “None” $\Rightarrow$ we report **macro-F1**.



One test wafer per class (dark = failed die).

The paper we reproduce

Kang & Kang, *Computers in Industry* 129 (2021) 103450 (Elsevier)

1. **MFE+FNN**: 59 handcrafted features (region densities, Radon transform, defect geometry) → small neural network
2. **CNN**: VGG16, ImageNet-pretrained, on the 64×64 map
3. **Stacking-MLR**: ridge regression on both models' class probabilities → a per-class weight for each model

Reported: Stacking-MLR macro-F1 **0.8949**, better than either base model and four other baselines.

Our contributions

1. **Full reproduction** of all six models, at four training-set sizes
2. **A meta-training pitfall** in the public code, and its fix
3. **Controlled extensions**: test-time augmentation, XGBoost, CNN seed ensembles
4. **Selective prediction**: send uncertain wafers to an engineer

Rules: all choices on a validation split B ; test set only for reporting; paired t -tests and wafer bootstrap intervals; negative results reported.

- Train 162,946 / test 10,000 (stratified)
- Training set $\rightarrow A$ (130,356): fit base models B (32,590): validation and meta-training
- $B = B_{\text{fit}} \cup B_{\text{val}}$; base models early-stop on B_{fit}
- Selection score: two-fold cross-fit of the stacker over B
- Inclusion rule: keep an extension if it beats the baseline by more than the run-to-run std

Reproduction at full data

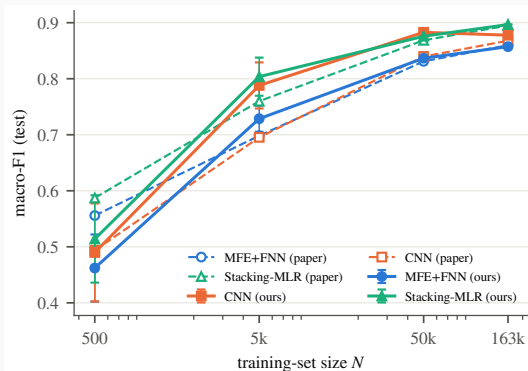
Model	macro-F1		micro-F1		runs
	ours	reported	ours	reported	
MFE+FNN	0.8572 ± 0.0062	0.8599 ± 0.0117	0.9735 ± 0.0003	0.9741 ± 0.0010	3 seeds
CNN	0.8777	0.8679 ± 0.0126	0.9790	0.9775 ± 0.0014	1 run
MultiNN	0.8880	0.8455 ± 0.0170	0.9794	0.9728 ± 0.0015	1 run
Stacking-DT	0.8782 ± 0.0065	0.8789 ± 0.0094	0.9752 ± 0.0005	0.9757 ± 0.0011	3 seeds
Stacking-FNN [†]	0.9041 ± 0.0028	0.8991 ± 0.0096	0.9811 ± 0.0002	0.9808 ± 0.0010	5 seeds
Stacking-MLR	0.8967 ± 0.0060	0.8949 ± 0.0121	0.9805 ± 0.0004	0.9801 ± 0.0010	3 seeds

All six models match or beat the paper. [†]Stacking-FNN fitted on B .

The meta-training pitfall

- Public code: stacker trained on the base models' predictions **for their own training data**
- There the models are almost always right and very confident $\Rightarrow$ the stacker learns “always follow them”
- Result: the rarest class (Near-full) is never predicted, macro-F1 0.6298 ± 0.0904
- Class weights only hide it (0.8402 ± 0.0393 , unstable)
- **Fix:** train the stacker on held-out data B — the classic advice for stacking
- Linear MLR is far less affected than FNN or tree stackers

Training-size sweep



- $N \geq 5,000$: every model beats the paper
- $N = 500$: noisy; rare classes often missing from 400 wafers
- $N = 50,000$: MLR stack below our CNN under the paper's in-sample protocol — the same pitfall

Configuration	B (MLR)	test (MLR)	test (FNN)
Stack-2: MFE-FNN + CNN	0.9030 ± 0.0034	0.9005 ± 0.0076	0.9004 ± 0.0048
+ TTA on the CNN	0.9074 ± 0.0045	0.9051 ± 0.0069	0.9067 ± 0.0062
+ XGBoost as third learner	0.9105 ± 0.0030	0.9149 ± 0.0025	0.9070 ± 0.0037
Control: XGBoost replaces MFE-FNN	0.9123	0.9160	0.9130 ± 0.0022
Control: 2nd MFE-FNN seed as third learner	0.9094 ± 0.0040	0.9059 ± 0.0051	0.9029 ± 0.0057
MFE-FNN + CNN $\times 5$	0.9165 ± 0.0057	0.9075 ± 0.0054	0.9095 ± 0.0029
MFE-FNN + CNN $\times 5$ + XGBoost	0.9182 ± 0.0039	0.9125 ± 0.0015	0.9065 ± 0.0034
CNN$\times 5$ + XGBoost (final)	0.9223	0.9134	0.9092 ± 0.0023

TTA: average over 8 rotations/flips CNN $\times 5$: five CNN seeds averaged

What helped, what did not

Helped

- Test-time augmentation (significant on B)
- XGBoost as the handcrafted-feature model
- **Five-CNN ensemble**: largest gain; seeds differ by ± 0.012 macro-F1 at equal loss

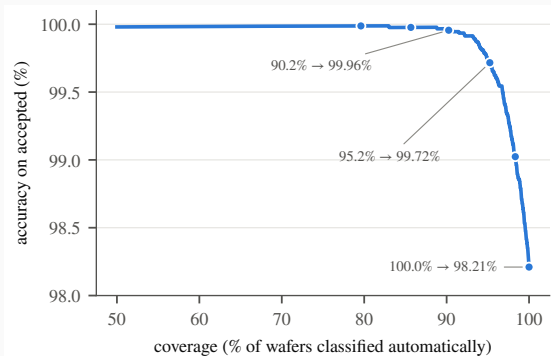
Did not help

- *A different* third model vs. a second seed
- The paper's MFE+FNN once the CNN is an ensemble
- Bagging XGBoost, class weights, mixed precision
- Temperature scaling for MLR calibration

5 CNNs (TTA) + XGBoost $\xrightarrow{\text{MLR}}$ class

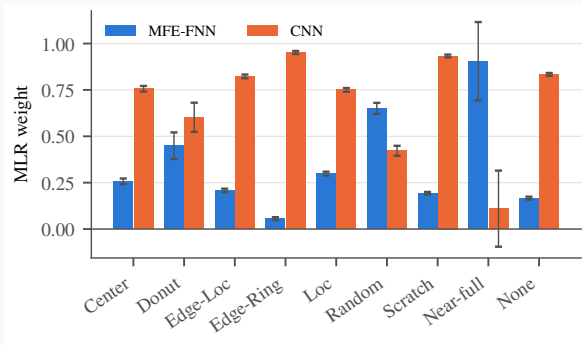
- Test macro-F1 **0.9134** (paper 0.8949, our reproduction 0.8967)
- Accuracy 98.21%
- Gain over reproduction +0.0168, bootstrap interval excludes zero
- Runnable from saved models: `predict.py`

Selective prediction



- Confidence = max probability (temperature-scaled)
- Thresholds chosen on B , applied to test
- 95.2% automatic at 99.72% accuracy
- Deferred: mostly Loc, Scratch, Edge-Loc; only 2% of None

What the stack learns



- Handcrafted features carry **Random** and **Near-full** (global density)
- CNN carries the spatial patterns (Edge-Ring, Scratch)
- Matches the paper's Fig. 5
- SHAP: eccentricity/solidity for Scratch, defect area for Near-full

Limitations

- One fixed train/test split (paper: ten)
- Near-full has 9 test wafers: one wafer ≈ 0.01 macro-F1
- Test set consulted twice (always after selection on B)
- Five replicates at $N = 50,000$; MultiNN single run; preliminary model-selection tables not repeated

Conclusion

- The paper reproduces: 0.8967 vs 0.8949
- Train the stacker on held-out outputs, never in-sample
- The handcrafted branch helps a single CNN but not a CNN ensemble
- Best: 0.9134 macro-F1; 95.2% of wafers at 99.72% accuracy with a reject option

Code: github.com/Kartikeya2046/ML_wafer_detecting_using_stack_ensemble